KAAN UZ
2022400195

ASSIGNMENT 2 REPORT

Video link : https://youtu.be/54fo0oh5Jiw

1. INTRODUCTION

This Is The Only Level is a nostalgic platformer where a player, represented by an elephant, must repeatedly navigate through a single level. However, each time the stage is re-entered, a unique twist or mechanic is introduced, challenging the player to adapt their movement strategy. The player must reach the exit pipe by solving platforming puzzles, pressing buttons, avoiding spikes, and handling changes in movement physics or controls. There are a total of five stages implemented, each offering a new gameplay experience while keeping the level visually the same.

2. GAME MECHANICS

COLLISIONS: In the game, collision detection ensures that the player cannot move through solid objects like obstacles, the door, or spikes. Each object (the player and the blocks) is treated as a rectangle, and collisions are checked by comparing their sides.

$$playerLeft = x - w / 2 \quad (1)$$

$$playerRight = x + w / 2 \quad (2)$$

$$playerTop = y + h / 2 \quad (3)$$

$$playerBottom = y - h / 2 \quad (4)$$

$$horizontalOverlap = playerRight > boxLeft \;\&\&\; playerLeft < boxRight \quad (5)$$

$$verticalOverlap = playerTop > boxBottom \;\&\&\; playerBottom < boxTop \quad (6)$$

PLAYER MOVE : The player moves based on keyboard input. When the left or right arrow key is pressed, the player's x position is updated accordingly. When the up arrow key is pressed and the player is on the ground, a jump is triggered. Gravity continuously affects the vertical velocity (velocityY) and causes the player to fall.

$$x = x \pm velocityX \quad (7)$$

velocityY += gravity (8)

y += velocityY (9)

INTERACTING WITH DOOR: In the game, the **button** and **door** system is essential for stage progression. The **door** initially blocks the exit pipe, and the player must **press the button** to open it. A button is considered "pressed" when the player stands on it. The game detects this using the same collision logic as obstacles.

- In most stages, pressing the button once opens the door.
- In **Stage 4**, the button must be pressed **five times** before the door opens.
- Once triggered, the door begins an **animated downward slide** to disappear, allowing the player to enter the exit pipe.

ICY FLOOR MOVEMENT (STAGE 5): In Stage 5, the floor becomes slippery. Instead of instantly stopping when the key is released, the player keeps sliding forward due to momentum. Acceleration is added incrementally when moving, and friction is simulated by reducing velocity over time. This creates a realistic **inertia-based movement** on ice.

if (rightKey) currentVX += velocityX * 0.2 (10)
if (leftKey) currentVX -= velocityX * 0.2 (11)
currentVX *= 0.95 (12)
x += currentVX (13)

PASSING THE STAGE MECHANICS: In the game, the player advances to the next stage by **entering the exit pipe**, but only **after the door has been opened**. This introduces a win condition that combines both movement and puzzle-solving: pressing the button(s) and reaching the exit without dying.

3. IMPLEMENTATION DETAILS

The game was implemented in Java using object-oriented programming principles. The graphical interface was built using the StdDraw library. The game consists of the following key classes:

3.1 STAGE CLASS

Encapsulates each game's stage mechanics, including gravity, movement keys, and stage hints. Each stage introduces new mechanics like reversed controls, bouncing movement, or icy surfaces. (as shown in Figure 1)

```java
// Data Fields
private int stageNumber;  2 usages
private double gravity;  2 usages
private double velocityX;  2 usages
private double velocityY;  2 usages
private int rightCode;  2 usages
private int leftCode;  2 usages
private int upCode;  3 usages
private String clue;  2 usages
private String help;  2 usages
private Color color;  2 usages
private Map map;  2 usages

// Constructor
public Stage(double gravity, double velocityX, double velocityY, int stageNumber,  5 usages
             int rightCode, int leftCode, int upCode, String clue, String help) {
    this.gravity = gravity;
    this.velocityX = velocityX;
    this.velocityY = velocityY;
    this.stageNumber = stageNumber;
    this.rightCode = rightCode;
    this.leftCode = leftCode;
    this.upCode = upCode;
    this.clue = clue;
    this.help = help;

    Random rand = new Random();
    this.color = new Color(rand.nextInt( bound: 256), rand.nextInt( bound: 256), rand.nextInt( bound: 256));
}
```

Figure 1: Stage Class

3.2 PLAYER CLASS

Handles the elephant's movement (as shown in 2), gravity application (as shown in 3), jumping logic, and death count. It manages facing direction based on movement and is responsible for drawing the player with appropriate images.

```java
public void move(char direction, Stage stage) {  1 usage
    double vx = stage.getVelocityX();
    double nextX;

    // Stage 5 - icy floor
    if (stage.getStageNumber() == 5) {
        if (direction == 'r') {
            currentVX += vx * 0.2;
        } else if (direction == 'l') {
            currentVX -= vx * 0.2;
        }

        // friction effect
        currentVX *= 0.95;

        nextX = x + currentVX;

        if (!stage.getMap().checkCollision(nextX, y, stage.getMap().getObstacles())) {
            x = nextX;
        }

        if (currentVX > 0.1) {
            facingRight = true;
        } else if (currentVX < -0.1) {
            facingRight = false;
        }
    }
```

Figure 2: Player Movement

```java
public void applyGravity(Stage stage, Map map) {  1 usage
    double gravity = stage.getGravity();
    velocityY += gravity;
    double maxY = 600;
    double step;
    if (velocityY > 0) {
        step = 1;
    }else if ((velocityY < 0)){
        step = -1;
    }else {
        step = 0;
    }

    int steps = (int) Math.abs(velocityY);

    for (int i = 0; i < steps; i++) {
        double nextY = y + step;

        if (nextY > maxY) {
            velocityY = 0;
            onGround = false;
            return;
        }
        if (!map.checkCollision(x, nextY, map.getObstacles())) {
            y = nextY;
            onGround = false;
            justBounced = false;
        } else {
            velocityY = 0;
            onGround = true;
```

Figure 3: Gravity

3.3 MAP CLASS

Manages the entire stage layout, including obstacle placement, spike collisions, button mechanics, door animation, and rendering all visual components. (shown in figure 5) It also contains the collision logic. (shown in figure 4)

```java
public boolean checkCollision(double x, double y, int[][] boxes) {  8 usages
    // check obstacle collision
    for (int[] box : boxes) {
        double playerLeft = x - player.getWidth() / 2;
        double playerRight = x + player.getWidth() / 2;
        double playerBottom = y - player.getHeight() / 2;
        double playerTop = y + player.getHeight() / 2;

        double boxLeft = box[0];
        double boxRight = box[2];
        double boxBottom = box[1];
        double boxTop = box[3];

        boolean horizontalOverlap = playerRight > boxLeft && playerLeft < boxRight;
        boolean verticalOverlap = playerTop > boxBottom && playerBottom < boxTop;

        if (horizontalOverlap && verticalOverlap) {
            return true;
        }
    }
```

Figure 4: Collision Detection

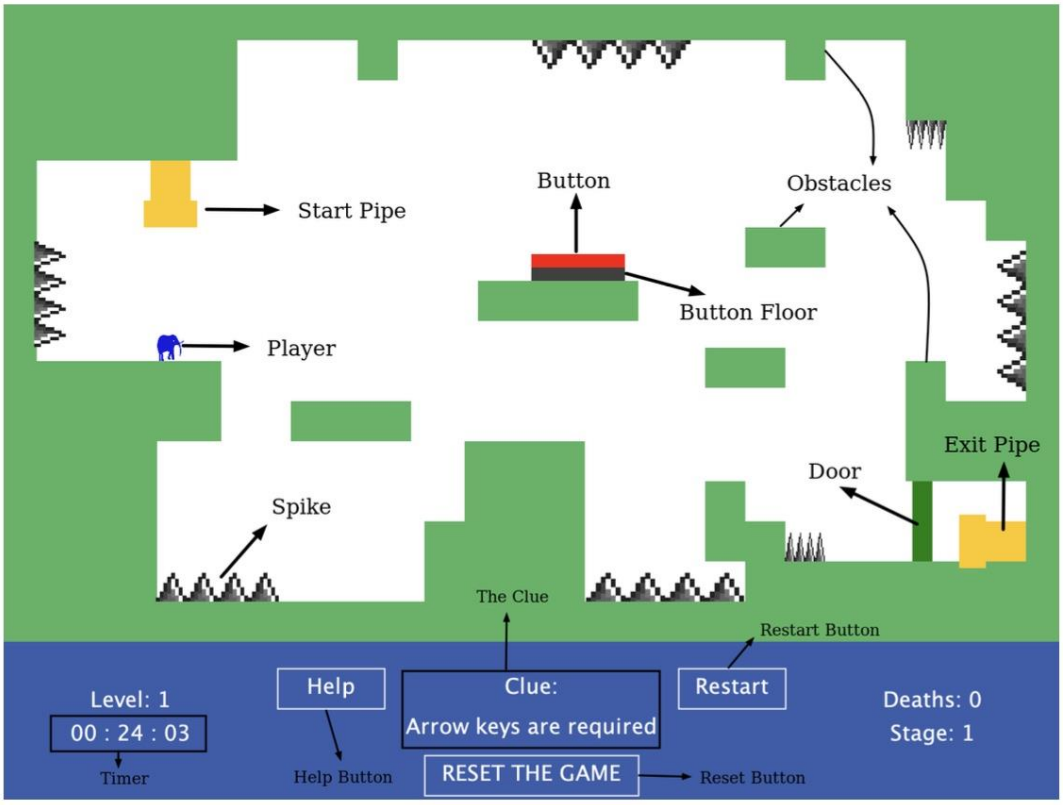

Figure 5 : Gameplay Screen Photo

3.4 GAME CLASS

Coordinates the main game loop (shown in figure 6), stage transitions, timer, user input (keyboard and mouse) (shown in figure 7), and death handling. It displays the final banner and manages game restarts.

```
public void play() {  2 usages
    wasMousePressed = false;
    StdDraw.enableDoubleBuffering();

    while (stageIndex < stages.size()) {
        Stage currentStage = stages.get(stageIndex);
        player = new Player( x: 115,  y: 450); // önce player oluştur

        map = new Map(currentStage, player); // sonra map oluştur
        currentStage.setMap(map);

        boolean stagePassed = false;

        while (!stagePassed) {
            if (resetGame) {
                stageIndex = 0;
                deathNumber = 0;
                gameTime = 0;
                resetTime = System.currentTimeMillis();

                player = new Player( x: 115,  y: 450);
                map = new Map(stages.get(stageIndex), player);
                resetGame = false;
                break;
            }

            StdDraw.clear();
            map.applyGravity();
            handleInput(map);
```

Figure 6: Stage Class

```java
private void handleInput(Map map) {  1 usage
    Stage s = map.getStage();
    int[] keys = s.getKeyCodes();

    if (keys[0] != -1 && StdDraw.isKeyPressed(keys[0])) {
        map.movePlayer( direction: 'r');
    }
    if (keys[1] != -1 && StdDraw.isKeyPressed(keys[1])) {
        map.movePlayer( direction: 'l');
    }
    if (keys[2] != -1 && StdDraw.isKeyPressed(keys[2])) {
        map.movePlayer( direction: 'u');
    }

    // Mouse controls
    double mx = StdDraw.mouseX();
    double my = StdDraw.mouseY();
    boolean isMousePressedNow = StdDraw.isMousePressed();

    boolean isInsideRestart = (mx >= 510 && mx <= 590 && my >= 70 && my <= 100);
    if (isMousePressedNow && !wasMousePressed && isInsideRestart) {
        map.getPlayer().incrementDeaths();
        map.restartStage();
    }
```

Figure 7: Handling Inputs

3.5 KAAN UZ (MAIN)

Initializes the game window and sets up the five stages (shown in figure 8), then starts the game by invoking Game.play().

```
// Stage 1: Normal controls
stages.add(new Stage( gravity: -0.45,  velocityX: 3.65,  velocityY: 10,  stageNumber: 1,
        KeyEvent.VK_RIGHT, KeyEvent.VK_LEFT, KeyEvent.VK_UP,
        clue: "Arrow keys are required",  help: "Arrow keys move player ,press button and enter the second pipe"));

// Stage 2: Reversed directions
stages.add(new Stage( gravity: -0.45,  velocityX: 3.65,  velocityY: 10,  stageNumber: 2,
        KeyEvent.VK_LEFT, KeyEvent.VK_RIGHT, KeyEvent.VK_UP,
        clue: "Not always straight forward",  help: "Right and left buttons reversed"));

// Stage 3: Constantly jumping
stages.add(new Stage( gravity: -2,  velocityX: 3.65,  velocityY: 24,  stageNumber: 3,
        KeyEvent.VK_RIGHT, KeyEvent.VK_LEFT,  upCode: -1,
        clue: "A bit bouncy here",  help: "You jump constantly"));

// Stage 4: press the button 5 times
stages.add(new Stage( gravity: -0.45,  velocityX: 3.65,  velocityY: 10,  stageNumber: 4,
        KeyEvent.VK_RIGHT, KeyEvent.VK_LEFT, KeyEvent.VK_UP,
        clue: "Never gonna give you up",  help: "Press button 5 times"));

// Stage 5: icy floor
stages.add(new Stage(
        gravity: -0.45,  velocityX: 3.65,  velocityY: 10,  stageNumber: 5,
        KeyEvent.VK_RIGHT, KeyEvent.VK_LEFT, KeyEvent.VK_UP,
        clue: "It's slippery!",  help: "Tap the keys gently. The floor is icy."));
```

Figure 8: Initializing Stages